SHOPSPHERE DEVOPS PROJECT

EC2 creation to CI/CD, automated rollback and local Prometheus/Grafana setup

Complete implementation and command reference. This guide explains what each command does, where to run it, why it is used, the verified outcomes and the remaining work.

Environment	Value
AWS region	ap-southeast-2 (Sydney)
EC2 instance	i-0af087445cc5f5878
Public IP at setup	13.239.24.127 (may change after stop/start)
OS / size	Ubuntu 24.04 / t3.small
Known-good release	build-24
ECR repositories	shopsphere-user, shopsphere-product, shopsphere-order, shopsphere-frontend
Windows monitoring folder	C:\DevOps-Monitoring
Ports	Jenkins 9090; Prometheus 9091; Grafana 3000; Node Exporter 9100 via SSM

Security: No AWS access keys, secret keys, JWT secrets or passwords are included. Never commit credentials or Terraform state to Git.

1. Architecture and project overview

Jenkins runs on Windows. It checks out source, validates configuration, runs tests, builds versioned Docker images, pushes them to Amazon ECR and deploys to EC2 through AWS Systems Manager (SSM). EC2 runs Ubuntu, Docker Compose, Nginx and ShopSphere services. Terraform provisions AWS infrastructure.

Component	Purpose
GitHub	Source control for application, Dockerfiles, Compose, Jenkinsfile and Terraform.
Jenkins	CI/CD orchestration on Windows.
ECR	Private registry for four versioned application images.
Terraform	Infrastructure as Code for EC2, security group, IAM role/profile and ECR references.
EC2 Ubuntu	Runs Nginx, frontend and three backend microservices.
SSM	Secure command execution and port forwarding; avoids inbound SSH.
Node Exporter	Exposes EC2 Linux host metrics on port 9100.
Prometheus / Grafana	Run natively on Windows for metrics collection and dashboards.

2. Terraform and AWS infrastructure

Set AWS CLI profile in Windows PowerShell

```
$env:AWS_PROFILE = "terraform"
$env:AWS_DEFAULT_REGION = "ap-southeast-2"
```

What: Selects the Terraform IAM profile and default AWS region for this PowerShell session. These environment variables do not persist after the terminal closes.

Terraform workflow commands

```
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
terraform output
```

Command	Simple explanation
terraform init	Downloads provider plugins and initializes the Terraform directory.
terraform validate	Checks Terraform syntax and configuration consistency.
terraform plan -out=tfplan	Creates a saved plan showing proposed infrastructure changes.
terraform apply tfplan	Applies the exact saved plan to AWS.
terraform output	Displays configured outputs such as EC2 ID and public IP.

Reported result: 6 resources added, 0 changed, 0 destroyed. Existing ECR repositories were referenced, not duplicated.

Terraform state hygiene

Do not commit terraform.tfstate, terraform.tfstate.backup or the .terraform directory. Commit .terraform.lock.hcl. Protect state because it can contain sensitive infrastructure data.

3. IAM, EC2 identity and SSM

EC2 uses role ShopSphere-EC2-Role and instance profile ShopSphere-EC2-Profile. Attached policies: AmazonEC2ContainerRegistryReadOnly and AmazonSSMManagedInstanceCore. This lets EC2 pull ECR images and communicate with SSM without long-lived AWS keys stored on the server.

Check current AWS identity

```
aws sts get-caller-identity
```

Displays the AWS identity used by the CLI. On EC2 it confirmed the assumed ShopSphere-EC2-Role.

Start an interactive SSM session (Windows PowerShell)

```
aws ssm start-session --target i-0af087445cc5f5878 --profile terraform --region ap-southeast-2
```

Opens a terminal to EC2 through Systems Manager. Avoids copying SSH keys and exposing inbound port 22.

SSM Run Command test

```
$CommandId = aws ssm send-command --instance-ids i-0af087445cc5f5878 --document-name
  AWS-RunShellScript --parameters 'commands=["echo SSM_DEPLOYMENT_WORKS"]' --region ap-southeast-2
  --query "Command.CommandId" --output text
aws ssm get-command-invocation --command-id $CommandId --instance-id i-0af087445cc5f5878 --region
  ap-southeast-2 --query "{Status:Status,Output:StandardOutputContent,Error:StandardErrorContent}"
  --output json
```

First line sends a shell command to EC2. Second line retrieves its status and output. The test returned Success.

Reliable JSON payload pattern

```
$f="$env:TEMP\shopsphere-command.json"
$b=@{DocumentName="AWS-RunShellScript";InstanceIds=@("i-0af087445cc5f5878");Parameters=@{commands=@("echo
  SSM_TEST_OK","hostname","docker compose ps")}}
[IO.File]::WriteAllText($f,($b|ConvertTo-Json -Depth 5),(New-Object System.Text.UTF8Encoding($false)))
$id=aws ssm send-command --cli-input-json "file://$f" --region ap-southeast-2 --query
  "Command.CommandId" --output text
Start-Sleep 8
aws ssm get-command-invocation --command-id $id --instance-id i-0af087445cc5f5878 --region
  ap-southeast-2 --query "{Status:Status,Output:StandardOutputContent,Error:StandardErrorContent}"
  --output json
```

Builds a UTF-8 JSON file without BOM, sends it to SSM and retrieves output. This pattern avoided complex PowerShell quoting problems.

Jenkins IAM

The Jenkins IAM user has ECR push permissions and SSM actions: SendCommand, GetCommandInvocation, ListCommandInvocations and DescribeInstanceInformation. The learning-stage SSM policy used Resource '*'; narrow this for production.

4. EC2 server preparation and checks

Ubuntu 24.04 EC2 was configured with Docker Engine, Docker Compose and AWS CLI v2. Application deployment directory: /opt/shopsphere.

```
sudo systemctl status docker --no-pager
docker --version
docker compose version
aws --version
sudo docker run hello-world
df -h /
free -h
docker stats --no-stream
sudo ss -tulpn
```

Command	Purpose
systemctl status docker	Checks whether Docker daemon is running.
docker --version / docker compose version	Displays Docker and Compose versions.
aws --version	Displays AWS CLI version.
docker run hello-world	Tests that Docker can pull and run a container.
df -h /	Shows root disk size and free space.
free -h	Shows RAM and swap usage.
docker stats --no-stream	Displays one-time container CPU/memory usage.
ss -tulpn	Lists listening ports and processes.

Inspect application on EC2

```
cd /opt/shopsphere
pwd
ls -la
cat image-tag.env
docker compose --env-file image-tag.env ps
```

Moves into the deployment directory, lists files, reads the active image tag and displays Compose status. Run in Ubuntu EC2 SSM terminal.

Resource check showed about 1.9 GiB total RAM, 1.2 GiB available and 15 GiB free disk. This is why Prometheus and Grafana were placed on Windows rather than EC2.

5. ECR and Docker image build/push

Registry: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com. Repositories: shopsphere-user, shopsphere-product, shopsphere-order and shopsphere-frontend. Jenkins uses unique tags such as build-24.

Build commands used in Jenkins (Windows batch)

```
docker build -t %ECR_REGISTRY%/shopsphere-user:%IMAGE_TAG% -f services/user-service/Dockerfile
  services/user-service
docker build -t %ECR_REGISTRY%/shopsphere-product:%IMAGE_TAG% -f services/product-service/Dockerfile
  services/product-service
docker build -t %ECR_REGISTRY%/shopsphere-order:%IMAGE_TAG% -f services/order-service/Dockerfile
  services/order-service
docker build -t %ECR_REGISTRY%/shopsphere-frontend:%IMAGE_TAG% -f frontend/Dockerfile frontend
```

Builds each Docker image using its Dockerfile and tags it with the full ECR URI and pipeline build tag.

ECR login and push

```
aws ecr get-login-password --region %AWS_REGION% | docker login --username AWS --password-stdin %ECR_REGISTRY%
docker push %ECR_REGISTRY%/shopsphere-user:%IMAGE_TAG%
docker push %ECR_REGISTRY%/shopsphere-product:%IMAGE_TAG%
docker push %ECR_REGISTRY%/shopsphere-order:%IMAGE_TAG%
docker push %ECR_REGISTRY%/shopsphere-frontend:%IMAGE_TAG%
```

Gets a temporary ECR login password, logs Docker in and pushes four images. Percent variables are Windows batch/Jenkins syntax.

```
docker compose config -q
```

Validates Compose configuration quietly; it does not start containers.

6. Application deployment and Nginx

Nginx path	Upstream
/	frontend:80
/api/users/	user-service:3001
/api/products/	product-service:3002
/api/orders/	order-service:3003

```
cd /opt/shopsphere
cat image-tag.env
docker compose --env-file image-tag.env config -q
docker compose --env-file image-tag.env ps
docker compose --env-file image-tag.env pull
docker compose --env-file image-tag.env up -d --remove-orphans
curl -fsS http://localhost/ > /dev/null
```

Command	Simple explanation
cat image-tag.env	Displays the release tag selected for Compose.
compose config -q	Validates rendered Compose configuration.
compose ps	Shows service/container states.
compose pull	Pulls the selected image versions.
compose up -d --remove-orphans	Starts/recreates services in background and removes orphan containers.
curl -fsS ...	Checks local Nginx homepage and fails on HTTP errors.

Backend health check logic

```
for service in user-service product-service order-service; do
  container_id=$(docker compose --env-file image-tag.env ps -q "$service")
  health=$(docker inspect --format='{{.State.Health.Status}}' "$container_id")
  echo "$service: $health"
done
```

Loops over the three backend services, finds each container ID and reads its Docker health status. Deployment requires each to be healthy. Verified all five containers running, three backends healthy and HTTP 200.

7. Jenkins CI/CD pipeline

Jenkins runs on Windows at http://localhost:8080. Job: E-Commerce-CI-CD. Stages: source checkout, version generation, environment preparation, Compose validation, application tests, Docker build, image verification, ECR push and EC2 deployment through SSM.

Credential binding example

```
withCredentials([usernamePassword(
  credentialsId: 'aws-ecr-jenkins',
  usernameVariable: 'AWS_ACCESS_KEY_ID',
  passwordVariable: 'AWS_SECRET_ACCESS_KEY'
)]) {
  // AWS CLI and ECR operations execute inside this block.
}
```

Temporarily injects the stored Jenkins credential into the build environment. Never echo or commit the values.

Deployment invocation

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File scripts\deploy-ssm.ps1 -ImageTag %IMAGE_TAG%
if errorlevel 1 exit /b 1
```

Runs the PowerShell deployment script from Jenkins. A nonzero script exit code fails the pipeline. Jenkins build and ECR push were reported successful.

8. Deployment script and automatic rollback

File: scripts\deploy-ssm.ps1. It accepts an image tag, constructs an SSM Run Command payload, writes JSON without BOM, submits the command, polls SSM status and fails the PowerShell script if the remote command fails.

Required parameter

```
param(
  [Parameter(Mandatory = $true)]
  [ValidatePattern('^build-[0-9]+$')]
  [string]$ImageTag
)
```

Requires a release tag in build-number format.

PowerShell syntax validation

```
$tokens=$null
$errors=$null
$path=Join-Path $PWD "deploy-ssm.ps1"
$ast=[System.Management.Automation.Language.Parser]::ParseFile($path,[ref]$tokens,[ref]$errors)
if ($errors.Count -eq 0) { "PowerShell syntax OK" } else { $errors | Format-List Message,Extent }
```

Parses the PowerShell file and reports syntax errors. It does not validate remote Bash behavior.

Remote deployment logic (conceptual excerpt)

```
set -Ee
cd /opt/shopsphere
PREVIOUS_TAG=$(sed -n 's/^IMAGE_TAG=//p' image-tag.env | head -n 1)
echo "Previous release: $PREVIOUS_TAG"
echo "IMAGE_TAG=NEW_TAG" > image-tag.env
aws ecr get-login-password --region ap-southeast-2 | docker login --username AWS --password-stdin
  507941515612.dkr.ecr.ap-southeast-2.amazonaws.com
docker compose --env-file image-tag.env pull
docker compose --env-file image-tag.env up -d --remove-orphans
sleep 20
curl -fsS http://localhost/ > /dev/null
```

Saves old release tag, sets new tag, authenticates to ECR, pulls images, updates services and checks homepage. The full script also checks backend health.

SSM multiline script issue and fix

Sending the entire multiline Bash script as one SSM command string caused _script.sh not found / exit 127. The working fix split it into individual non-empty command lines:

```
Parameters = @{
  commands = @($RemoteScript -split "\r?\n" | Where-Object { $_.Trim().Length -gt 0 })
}
```

After this fix, normal deployment with build-24 succeeded.

Controlled rollback test

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\deploy-ssm.ps1 -ImageTag build-9999999
```

A deliberately nonexistent tag caused ECR image pull to fail. The ERR trap restored build-24, pulled previous images, restarted services and checked homepage plus backend health. Output included ROLLBACK_COMPLETED: build-24. SSM status Failed was expected because the new release failed; recovery succeeded.

Final verification command

```
$f="$env:TEMP\shopsphere-final-check.json"
$b=@{DocumentName="AWS-RunShellScript";InstanceIds=@("i-0af087445cc5f5878");Parameters=@{commands=@("cd
  /opt/shopsphere","cat image-tag.env","docker compose --env-file image-tag.env ps","curl -s -o
  /dev/null -w 'HTTP Status: %{http_code}\n' http://localhost/")}}
[IO.File]::WriteAllText($f,($b|ConvertTo-Json -Depth 5),(New-Object System.Text.UTF8Encoding($false)))
$id=aws ssm send-command --cli-input-json "file://$f" --region ap-southeast-2 --query
  "Command.CommandId" --output text
Start-Sleep 8
aws ssm get-command-invocation --command-id $id --instance-id i-0af087445cc5f5878 --region
  ap-southeast-2 --query "{Status:Status,Output:StandardOutputContent,Error:StandardErrorContent}"
  --output json
```

Final result: SSM Success, IMAGE_TAG=build-24, five containers running, three backend services healthy and HTTP 200.

9. Node Exporter installation on EC2

```
sudo apt update && sudo apt install -y prometheus-node-exporter
sudo systemctl enable --now prometheus-node-exporter
sudo systemctl status prometheus-node-exporter --no-pager
curl -fsS http://localhost:9100/metrics | head -n 10
```

Command	Purpose
sudo apt update	Refreshes Ubuntu package metadata.
apt install prometheus-node-exporter	Installs the Linux host metrics exporter.
systemctl enable --now	Enables startup at boot and starts service now.
systemctl status	Checks service state and logs.
curl .../metrics | head	Reads first ten lines of exported metrics.

Verified: service active/running and enabled; memory around 7.3 MB; metrics available on localhost:9100. curl may show error 23 when head closes the pipe after ten lines; metrics were returned.

10. Prometheus and Grafana on Windows (native install)

The user chose native Windows installation, not Docker. Jenkins uses 9090, so Prometheus uses 9091. Grafana uses 3000. EC2 Node Exporter uses 9100 through SSM port forwarding.

Create monitoring folders

```
mkdir C:\DevOps-Monitoring -Force
mkdir C:\DevOps-Monitoring\prometheus -Force
cd C:\DevOps-Monitoring
```

Creates a local workspace and Prometheus configuration folder.

Locate Prometheus executable

```
Get-ChildItem C:\DevOps-Monitoring -Filter prometheus.exe -Recurse
```

Found: C:\DevOps-Monitoring\prometheus\prometheus-3.16.0.windows-amd64\prometheus.exe.

Prometheus YAML configuration

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "ec2-node-exporter"
    static_configs:
      - targets: ["127.0.0.1:9100"]
```

Prometheus scrapes local loopback port 9100 every 15 seconds. The SSM tunnel will forward that local port to EC2 Node Exporter.

Write YAML with a PowerShell here-string

```
@'
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "ec2-node-exporter"
    static_configs:
      - targets: ["127.0.0.1:9100"]
'@ | Set-Content -Encoding ascii .\prometheus\prometheus.yml
```

The @' and '@ delimiters must each be on their own line. Pasting YAML on one line caused a PowerShell parser error.

Start Prometheus on 9091

```
& "C:\DevOps-Monitoring\prometheus\prometheus-3.16.0.windows-amd64\prometheus.exe"
  --config.file="C:\DevOps-Monitoring\prometheus\prometheus.yml"
  --web.listen-address="127.0.0.1:9091"
```

Starts Prometheus with its config and binds the web UI to localhost:9091. Keep the PowerShell window open.

Grafana ZIP distribution

Grafana Enterprise archive was extracted to C:\DevOps-Monitoring\grafana. It contains bin, conf, data, docs, packaging, plugins-bundled, public and tools. It is a ZIP distribution, not an MSI Windows service.

```
cd C:\DevOps-Monitoring\grafana\bin
.\grafana.exe server --homepath "C:\DevOps-Monitoring\grafana"
```

Starts Grafana 13.3.1. The 'server' subcommand is required by this Grafana CLI. Keep the terminal open. UI: http://localhost:3000. First login commonly uses admin/admin and prompts for a password change; choose a private password.

Secure SSM port forwarding (next action)

```
aws ssm start-session --target i-0af087445cc5f5878 --document-name AWS-StartPortForwardingSession
  --parameters '{"portNumber":["9100"],"localPortNumber":["9100"]}' --region ap-southeast-2
```

Forwards laptop port 9100 securely to EC2 port 9100. Keep the session open. No public security-group rule is needed. Tunnel and Prometheus target had not yet been verified at document time.

11. Ports and endpoints

Service	Endpoint	Status
Jenkins	localhost:9090	Existing Jenkins; reserved.
Prometheus	localhost:9091	Started locally; target check pending.
Grafana	localhost:3000	Started locally.
Node Exporter	EC2 localhost:9100	Installed; access via SSM tunnel.
ShopSphere	EC2 HTTP port 80	Previously returned HTTP 200.

Keep Prometheus and Grafana bound to localhost. Do not expose their ports publicly or add broad inbound security-group rules.

12. Troubleshooting encountered

Problem	Cause / fix
SSM _script.sh not found / exit 127	Multiline Bash sent as one string. Split into individual non-empty SSM commands.
PowerShell YAML parser error	Here-string pasted on one line. Put @' and '@ on separate lines.
prometheus.exe not found	Executable was nested in extracted folder. Search recursively and use exact path.
Prometheus port conflict	Jenkins owns 9090. Prometheus moved to 9091.
Grafana -homepath flag error	Grafana 13 needs 'server' subcommand before server flags.
PowerShell [ref] error	Initialize $tokens=$null and $errors=$null before ParseFile.
curl error 23 with head	head closed the pipe after ten lines; metrics were still returned.

13. Current status and next-session checklist

Workstream	Status at 10 Nov 2026
Terraform / EC2 / IAM / SG	Applied and created.
Docker / Compose / Nginx / ShopSphere	Running; build-24 verified.
ECR push through Jenkins	Successful.
Jenkins SSM deployment	Successful normal deployment.
Automatic rollback	Controlled missing-image test passed; build-24 restored.
Node Exporter	Installed, enabled, active; metrics verified.
Prometheus Windows	Downloaded/extracted and started on 9091.
Grafana Windows	Extracted and started on 3000.
SSM port forwarding	Command prepared; tunnel verification pending.
Prometheus target UP	Pending.
Grafana data source	Pending.
Linux dashboard / alerts	Pending.
cAdvisor container metrics	Not implemented.
Failure simulation / screenshots / final README	Pending.

Continue tomorrow in this order

```
1. Start EC2 in AWS Console; wait for status checks and SSM Online.
2. Verify app:
   cat /opt/shopsphere/image-tag.env
```

```
   cd /opt/shopsphere && docker compose --env-file image-tag.env ps
   curl -I http://localhost/
3. Verify Node Exporter:
   sudo systemctl status prometheus-node-exporter --no-pager
   curl -fsS http://localhost:9100/metrics | head -n 10
4. Start SSM port forwarding in a separate Windows PowerShell window.
5. Test tunnel from Windows:
   Invoke-WebRequest http://127.0.0.1:9100/metrics -UseBasicParsing
6. Ensure Prometheus is running on localhost:9091.
7. Open http://localhost:9091/targets; confirm ec2-node-exporter is UP.
8. In Grafana add Prometheus data source URL http://localhost:9091.
9. Save & test; import a Linux Node Exporter dashboard.
10. Capture dashboard and failure/recovery evidence for assignment.
```

If EC2 was stopped, its public IP may change. SSM uses the instance ID, so the tunnel command still targets the same instance. Confirm SSM Online before starting the tunnel.

Stopping EC2 and cost

Stopping EC2 stops instance compute charges while stopped, but EBS root volume and ECR image storage continue to incur charges. An Elastic IP, if allocated, may incur charges depending on association. Stop local Prometheus/Grafana with Ctrl+C; keep their files. Do not delete Terraform resources or ECR repositories just to pause.

Known-good checkpoint: build-24; five containers running; three backend services healthy; HTTP 200; rollback test completed.

14. DevOps concepts learned

Concept	Meaning in this project
Infrastructure as Code	Terraform defines repeatable AWS resources.
Least privilege	Separate EC2 role and Jenkins permissions; avoid storing AWS keys on EC2.
Versioned images	Unique build tags identify releases.
CI/CD	Jenkins validates, tests, builds, pushes and deploys.
Health checks	Homepage and backend health determine deployment success.
Rollback	Failed release restores previous known-good image tag.
Secure operations	SSM replaces inbound SSH and provides port forwarding.
Observability	Node Exporter exposes metrics; Prometheus scrapes; Grafana visualizes.

This guide distinguishes completed/verified tasks from prepared but pending integrations. It records commands and results shared in the project conversation; it does not claim pending monitoring steps are complete.